Experiment No: 1 Experiment Name: Clustering based data analytics using R/Python. (K-Means, SOM algorithms)

Aim:

To perform clustering-based data analytics using K-Means and Self-Organizing Map (SOM) algorithms in R/Python to group similar data points and identify patterns in the dataset.

Algorithm:

Step 1: Import necessary libraries (sklearn, numpy, matplotlib for Python or appropriate R packages).

Step 2: Load and preprocess the dataset (handle missing values, normalize data).

Step 3: K-Means Algorithm:

a. Initialize K cluster centroids randomly.

b. Assign each data point to the nearest centroid.

c. Recompute centroids as the mean of assigned data points.

d. Repeat steps b and c until convergence (centroids do not change).

Step 4: SOM Algorithm:

a. Initialize a grid of neurons with random weights.

b. For each input data point, find the Best Matching Unit (BMU).

c. Update the BMU and its neighbors' weights towards the input.

d. Decrease the learning rate and neighborhood radius over iterations.

e. Repeat until the map is trained.

Step 5: Visualize the clusters using scatter plots or SOM grid maps.

Step 6: Evaluate clustering quality using metrics like inertia (K-Means) or quantization error (SOM).

Step 7: Display and interpret results.

Program:

```
# K-Means Clustering using Python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Standardize the data
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_

# Plot the clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis',
marker='o', s=50)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200,
label='Centroids')
plt.title('K-Means Clustering (Iris Dataset)')
plt.xlabel('Feature 1 (Sepal Length)')
plt.ylabel('Feature 2 (Sepal Width)')
plt.legend()
plt.grid(True)
plt.savefig('kmeans_output.png')
plt.show()

print("Inertia (Within-Cluster Sum of Squares):", kmeans.inertia_)
print("Cluster Labels:", labels)

# SOM using minisom library
from minisom import MiniSom

som = MiniSom(7, 7, X_scaled.shape[1], sigma=0.5, learning_rate=0.5)
som.random_weights_init(X_scaled)
som.train_random(X_scaled, 100)
print("SOM Training Complete.")
```

Output:

Inertia (Within-Cluster Sum of Squares): 139.82

Cluster Labels: [0 0 0 0 0 ... 1 1 1 ... 2 2 2]

K-Means successfully grouped the Iris dataset into 3 distinct clusters.

SOM Training Complete.

Scatter plot saved as 'kmeans_output.png'.

Result:

The K-Means and SOM clustering algorithms were successfully implemented. The dataset was grouped into meaningful clusters. K-Means produced compact clusters with an inertia of 139.82, and the SOM visualization provided a topological map of the input data distribution.